

app

September 29, 2026

1 IND320 Project, assignment 1

By: Ole Andreas Grøtting

Github: [IND320 - OLEGRO99](#)

Streamlit: <https://ind320-olegro99-grxpvhtmnnnumfeeuqe8u7q.streamlit.app/>

1.0.1 Imports

```
[1]: import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt
import numpy as np
```

```
[2]: %matplotlib inline
```

1.0.2 load the dataset

```
[3]: reservoirs = pd.read_csv("data/reservoirs.csv", sep = ",")
reservoirs.head()
#we can see that some of the dates in "neste publiseringsdato" are set to 1 of
↪january year 1.
```

```
[3]:      dato_Id  omrType  omnr  iso_aar  iso_uke  fyllingsgrad  kapasitet_TWh  \
0  1995-09-03    EL      4    1995      35      0.944721      21.079208
1  2020-11-15    EL      1    2020      46      0.974361       6.003264
2  2011-08-28    EL      4    2011      34      0.786499      21.079208
3  1998-07-05    EL      3    1998      27      0.793969       8.920932
4  2011-03-27    EL      4    2011      12      0.300820      21.079208
```

```
      fylling_TWh  neste_Publiseringsdato  fyllingsgrad_forrige_uke  \
0      19.913979      0001-01-01T00:00:00      0.936888
1       5.849344      2020-11-25T13:00:00      0.997406
2      16.578772      0001-01-01T00:00:00      0.787193
3       7.082947      0001-01-01T00:00:00      0.733812
4       6.341054      0001-01-01T00:00:00      0.311122
```

```
      endring_fyllingsgrad
0              0.007833
```

```

1          -0.023046
2          -0.000694
3           0.060157
4          -0.010301

```

1.1 data cleaning

```

[4]: reservoirs = reservoirs.rename(columns={"dato_Id": "Date",
                                           "omrType": "Region",
                                           "omrn timer": "Region_Number",
                                           "iso_aar": "iso_year",
                                           "iso_uke": "iso_week",
                                           "fyllingsgrad": "filling_level",
                                           "kapasitet_TWh": "Capacity(TWh)",
                                           "fylling_TWh": "Filled (TWh)",
                                           "neste_Publiseringdato": "Next_Publication_Date",
                                           "fyllingsgrad_forrige_uke": "filling_level_previous_week",
                                           "endring_fyllingsgrad": "change_filling_level"})
reservoirs.head()

```

```

[4]:      Date Region Region_Number iso_year iso_week filling_level \
0  1995-09-03    EL              4    1995      35      0.944721
1  2020-11-15    EL              1    2020      46      0.974361
2  2011-08-28    EL              4    2011      34      0.786499
3  1998-07-05    EL              3    1998      27      0.793969
4  2011-03-27    EL              4    2011      12      0.300820

```

```

      Capacity(TWh) Filled (TWh) Next_Publication_Date \
0      21.079208      19.913979  0001-01-01T00:00:00
1       6.003264       5.849344  2020-11-25T13:00:00
2      21.079208      16.578772  0001-01-01T00:00:00
3       8.920932       7.082947  0001-01-01T00:00:00
4      21.079208       6.341054  0001-01-01T00:00:00

```

```

      filling_level_previous_week change_filling_level
0              0.936888              0.007833
1              0.997406             -0.023046
2              0.787193             -0.000694
3              0.733812              0.060157
4              0.311122             -0.010301

```

```

[5]:

```

```
reservoirs["Date"] = pd.to_datetime(reservoirs["Date"], errors='coerce')
reservoirs['Next_Publication_Date'] = pd.
↳to_datetime(reservoirs['Next_Publication_Date'], errors='coerce')
reservoirs.loc[
    reservoirs["Next_Publication_Date"].dt.year == 1,
    "Next_Publication_Date"
] = pd.NaT # Replace year 1 with NaT
reservoirs = reservoirs.dropna(subset=["Next_Publication_Date"])
# removing the rows with the year 1 in "neste publiseringsdato" and dropping
↳the rows with NaT values in "neste publiseringsdato"
reservoirs.head()
```

```
[5]:
```

	Date	Region	Region_Number	iso_year	iso_week	filling_level	\
1	2020-11-15	EL	1	2020	46	0.974361	
9	2023-08-13	EL	5	2023	32	0.836849	
12	2021-04-18	EL	5	2021	15	0.246682	
13	2019-06-02	EL	1	2019	22	0.659981	
15	2019-10-27	EL	1	2019	43	0.896057	

	Capacity(TWh)	Filled (TWh)	Next_Publication_Date	\
1	6.003264	5.849344	2020-11-25 13:00:00	
9	17.390587	14.553301	2023-08-23 13:00:00	
12	17.390587	4.289936	2021-04-28 13:00:00	
13	6.003264	3.962040	2019-06-12 13:00:00	
15	6.003264	5.379270	2019-11-06 13:00:00	

	filling_level_previous_week	change_filling_level
1	0.997406	-0.023046
9	0.776765	0.060084
12	0.276825	-0.030144
13	0.625398	0.034583
15	0.882742	0.013315

```
[6]: reservoirs.info()
reservoirs.describe()
```

```
<class 'pandas.DataFrame'>
Index: 3555 entries, 1 to 14876
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                  3555 non-null   datetime64[us]
1   Region                3555 non-null   str
2   Region_Number         3555 non-null   int64
3   iso_year              3555 non-null   int64
4   iso_week              3555 non-null   int64
5   filling_level         3555 non-null   float64
6   Capacity(TWh)         3555 non-null   float64
```

```

7   Filled (TWh)                3555 non-null   float64
8   Next_Publication_Date        3555 non-null   datetime64[us]
9   filling_level_previous_week  3555 non-null   float64
10  change_filling_level         3555 non-null   float64
dtypes: datetime64[us](2), float64(5), int64(3), str(1)
memory usage: 343.0 KB

```

```

[6]:
count      Date  Region_Number  iso_year  iso_week \
mean  2022-11-27 00:00:00      2.333333  2022.405063  26.187342
min    2019-02-17 00:00:00      0.000000  2019.000000   1.000000
25%    2021-01-03 00:00:00      1.000000  2020.000000  14.000000
50%    2022-11-27 00:00:00      2.000000  2022.000000  26.000000
75%    2024-10-20 00:00:00      3.000000  2024.000000  39.000000
max    2026-09-06 00:00:00      5.000000  2026.000000  53.000000
std                      NaN      1.490922   2.203409  14.727589

count  filling_level  Capacity(TWh)  Filled (TWh)  Next_Publication_Date \
mean    0.612520      29.145860    17.859596  2022-12-07 13:21:52.405063
min     0.080855       6.003264     0.485396   2019-02-27 13:00:00
25%     0.458974      17.390587     7.312559   2021-01-13 13:00:00
50%     0.638860      23.237715    14.420002   2022-12-07 13:00:00
75%     0.785713      34.043590    21.913880   2024-10-30 13:00:00
max     1.020072      87.437580    83.699460   2026-09-16 13:00:00
std     0.209919      22.741505    15.566634                      NaN

count  filling_level_previous_week  change_filling_level
mean    0.612048                    0.000473
min     0.080855                    -0.061430
25%     0.457667                    -0.021610
50%     0.637976                    -0.006449
75%     0.785222                    0.016729
max     1.020037                    0.195565
std     0.209927                    0.031425

```

1.2 plotting the data

```

[7]: #Plotting numerical columns
reservoirs = reservoirs.sort_values("Date")

reservoirs.plot(
    x="Date",
    y="filling_level",
    figsize=(12, 5)
)

```

```

plt.title("Filling level")
plt.xlabel("Date")
plt.ylabel("Filling level")
plt.show()

reservoirs = reservoirs.sort_values("Date")

reservoirs.plot(
    x="Date",
    y="Capacity(TWh)",
    figsize=(12, 5)
)

plt.title("Capacity(TWh)")
plt.show()

reservoirs = reservoirs.sort_values("Date")

reservoirs.plot(
    x="Date",
    y="Filled (TWh)",
    figsize=(12, 5)
)

plt.title("Filled (TWh)")
plt.show()

reservoirs = reservoirs.sort_values("Date")

reservoirs.plot(
    x="Date",
    y="filling_level_previous_week",
    figsize=(12, 5)
)

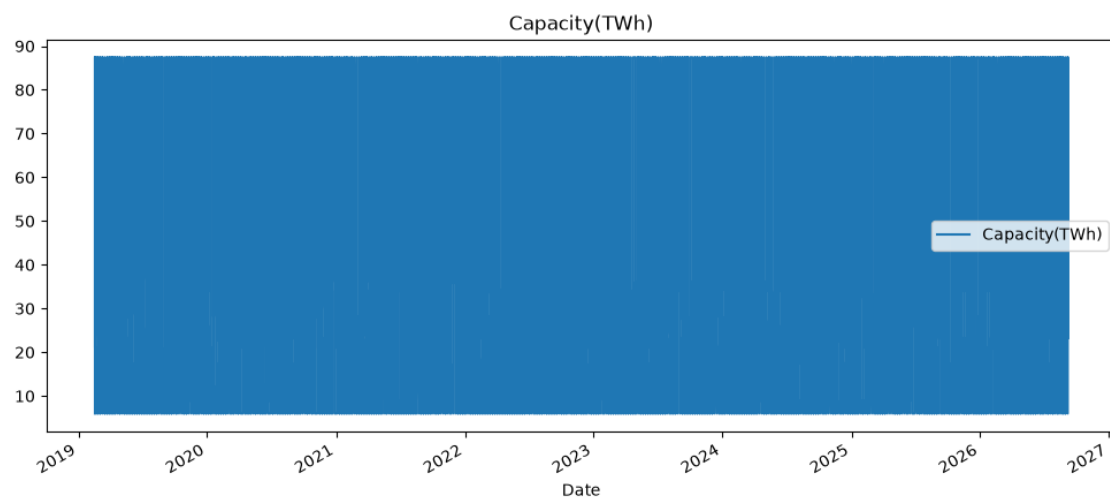
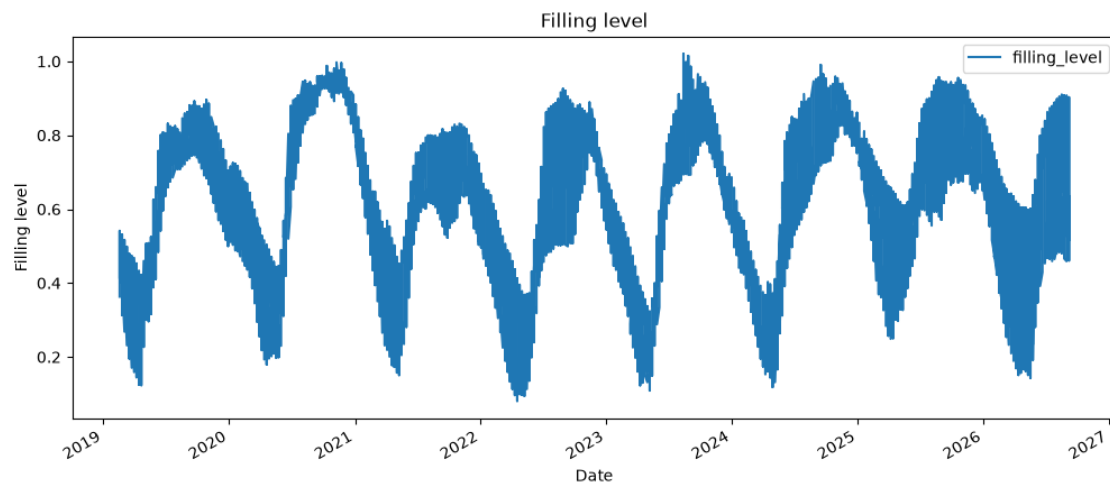
plt.title("Filling level previous week")
plt.show()

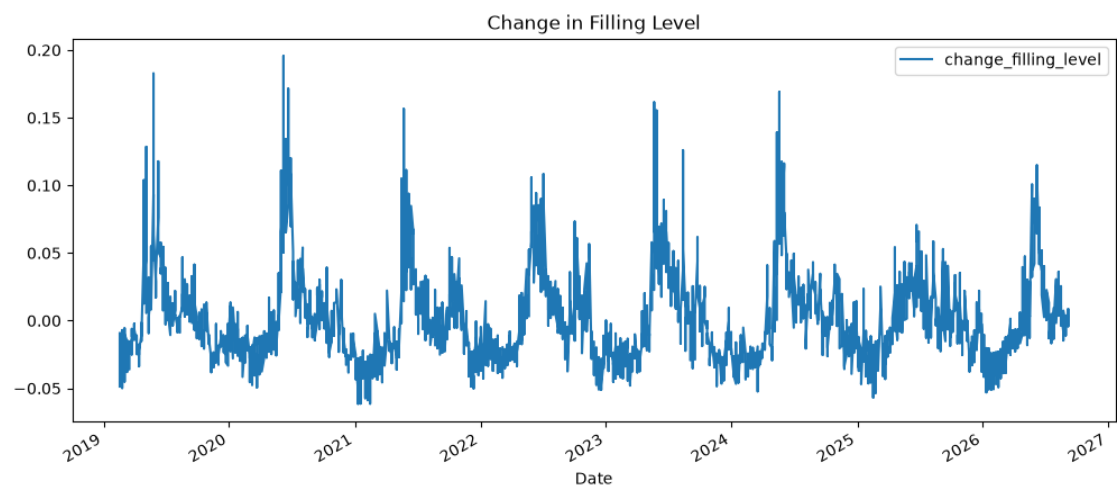
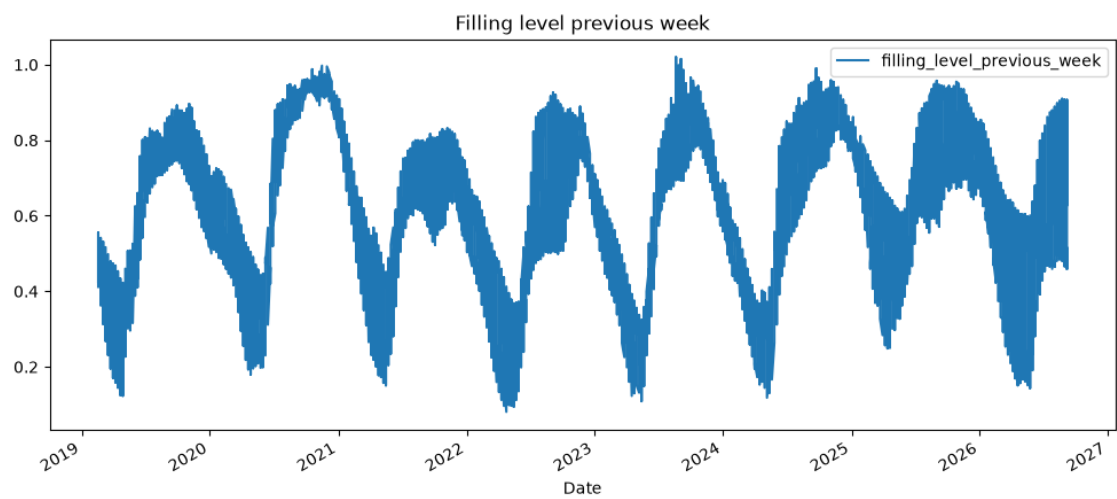
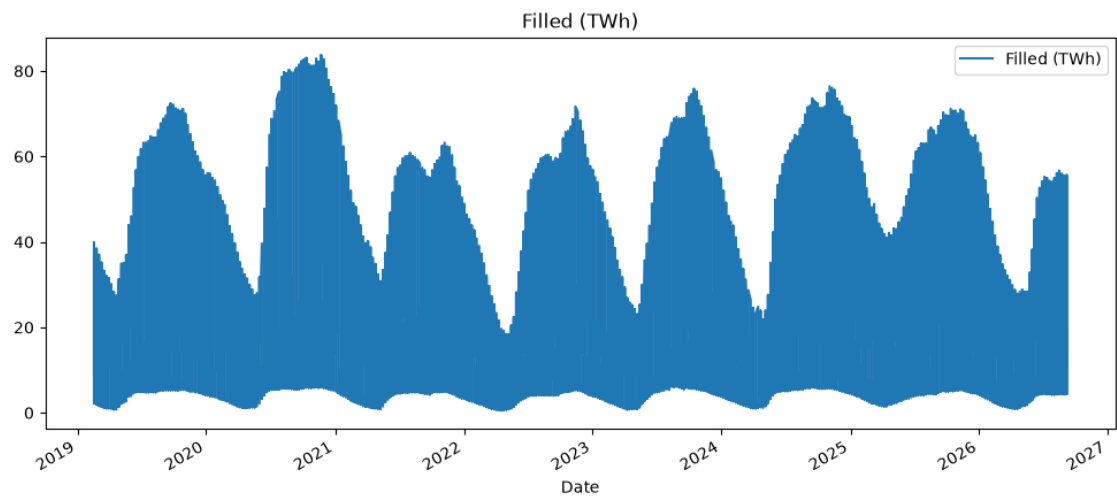
reservoirs = reservoirs.sort_values("Date")

reservoirs.plot(
    x="Date",
    y="change_filling_level",
    figsize=(12, 5)
)

```

```
plt.title("Change in Filling Level")  
plt.show()
```





Capacity is a box because the capacity is constant for every reservoir.

```
[13]: #plotting categorical columns
reservoirs["Region_Number"].value_counts().plot(kind="bar")

plt.title("Distribution of Region_Number")
plt.xlabel("Region_Number")
plt.ylabel("Count")
plt.show()

reservoirs["Region"].value_counts().plot(kind="bar")

plt.title("Distribution of Region")
plt.xlabel("Region")
plt.ylabel("Count")
plt.show()

plt.figure(figsize=(12, 5))

reservoirs["iso_year"].value_counts().sort_index().plot(kind="bar")

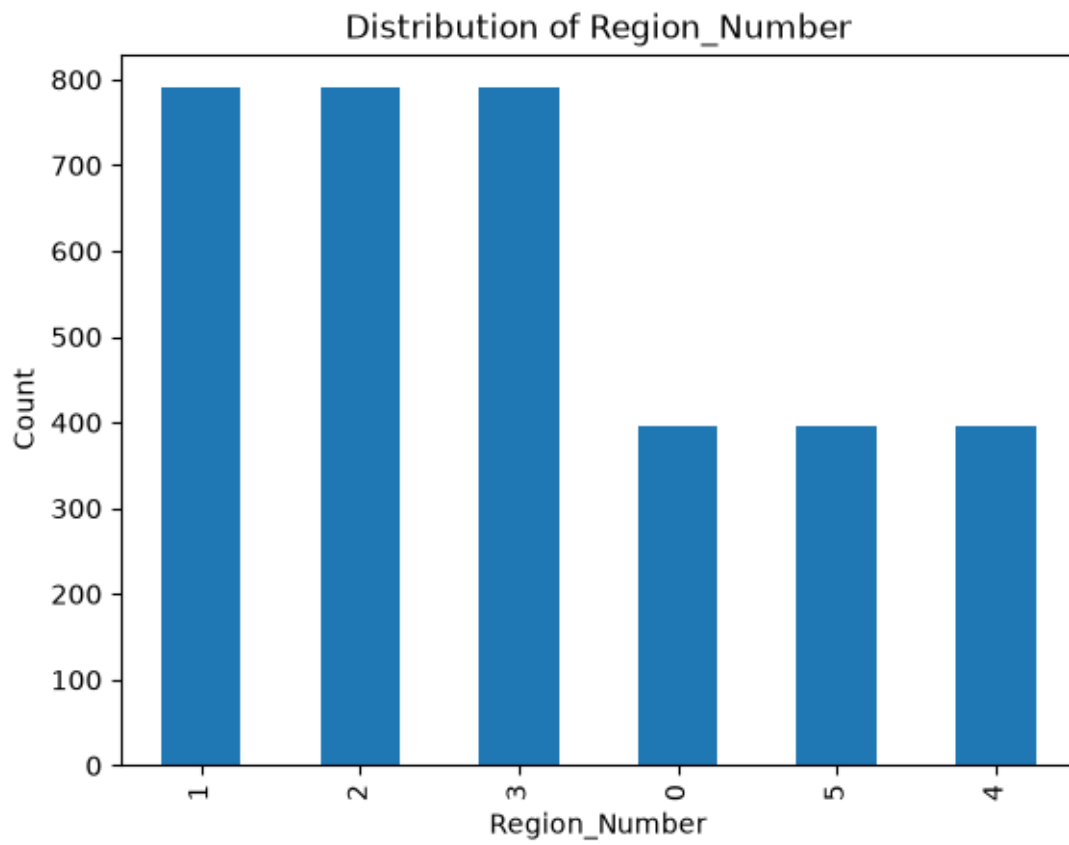
plt.title("Distribution of iso_year")
plt.xlabel("Year")
plt.ylabel("Count")
plt.show()

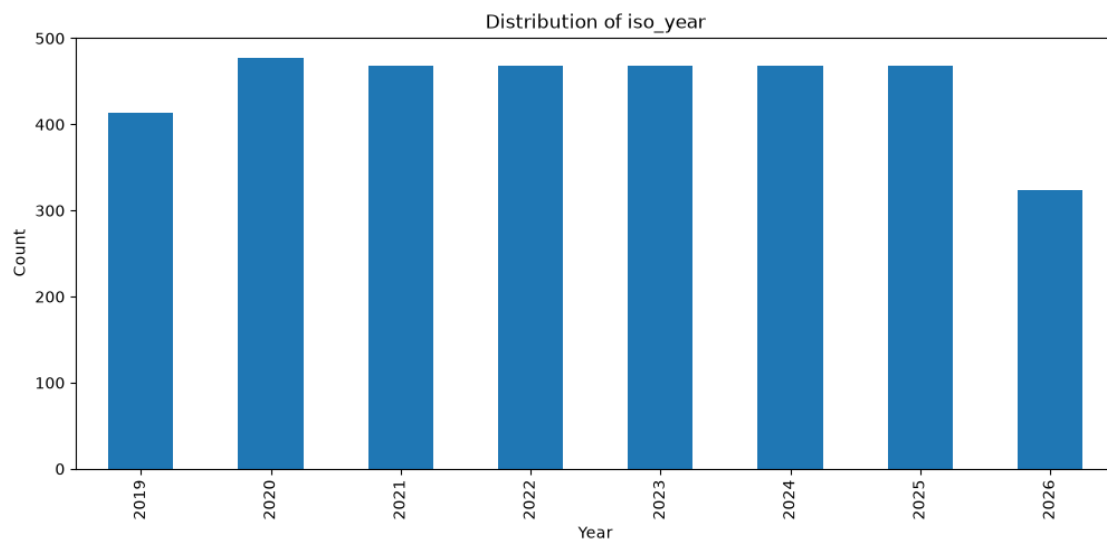
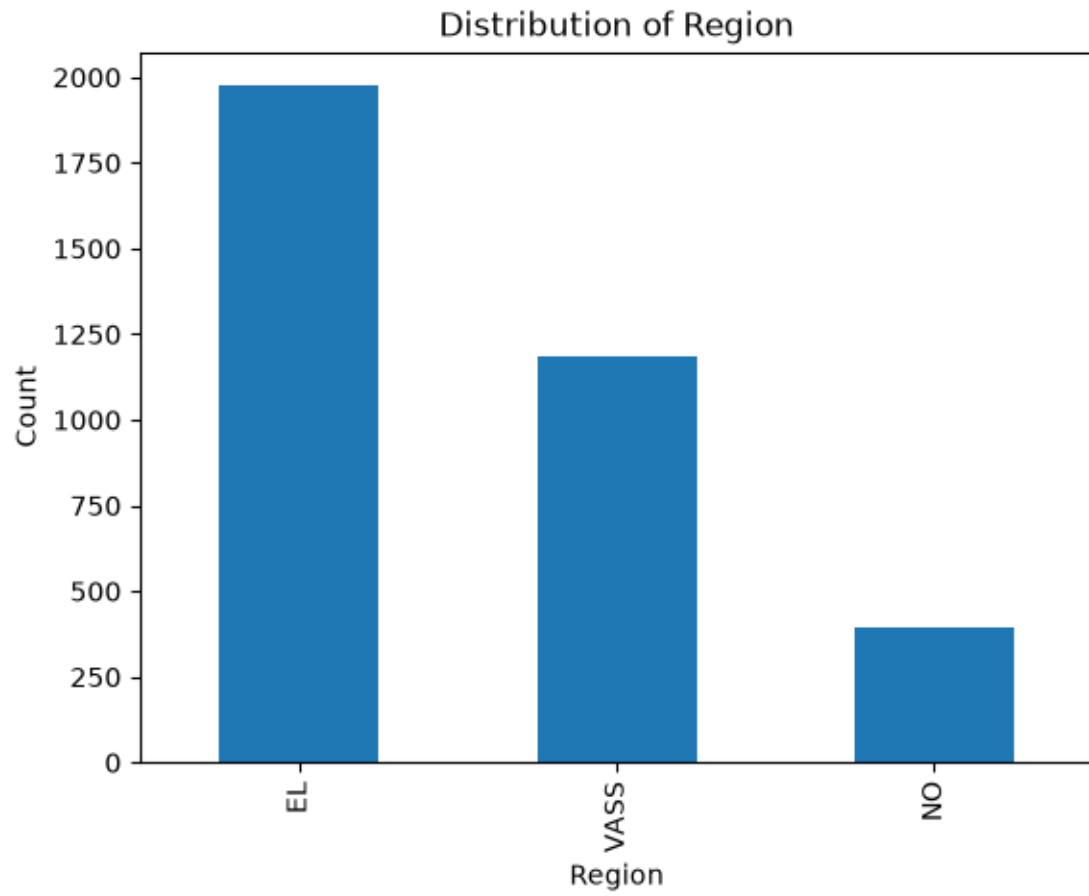
plt.figure(figsize=(12, 5))

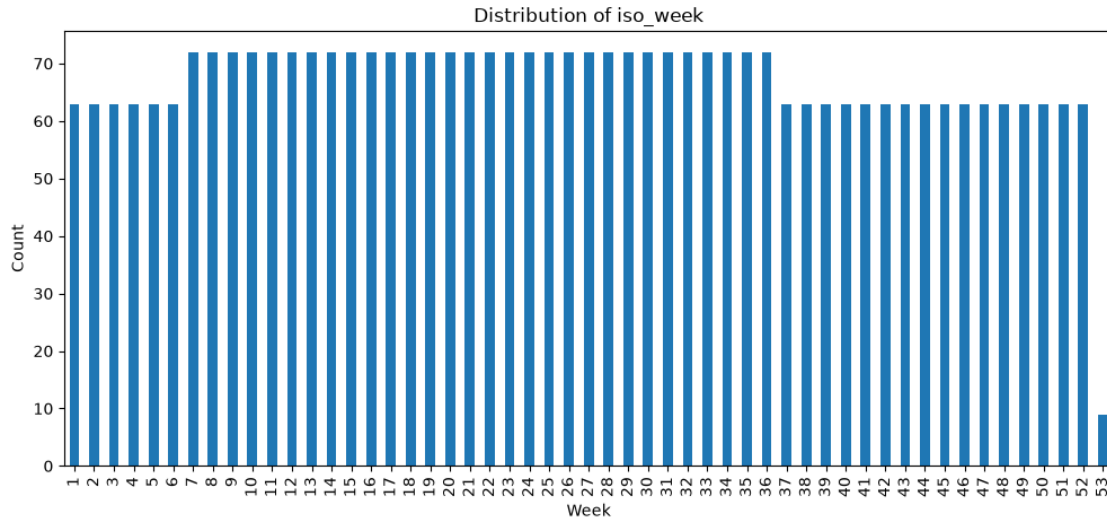
reservoirs["iso_week"].value_counts().sort_index().plot(kind="bar")

plt.title("Distribution of iso_week")
plt.xlabel("Week")
plt.ylabel("Count")

plt.show()
```



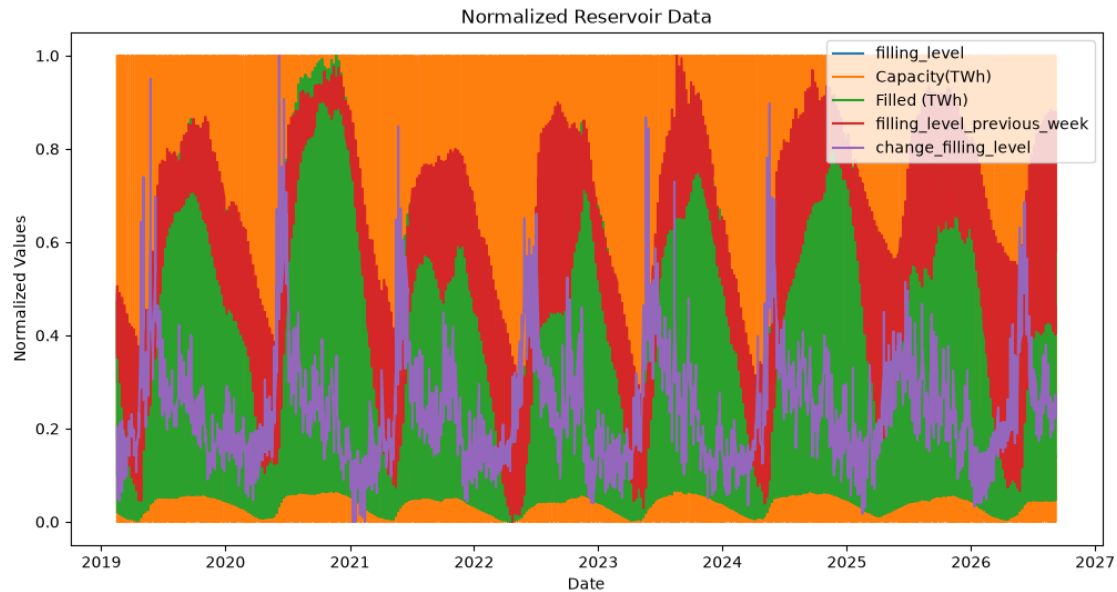




1.2.1 plotting all columns in the same plot

```
[10]: #min max normalize
reservoirs_norm = reservoirs.copy()
numerical_columns = ["filling_level", "Capacity(TWh)", "Filled (TWh)",
    ↪ "filling_level_previous_week", "change_filling_level"]
for col in numerical_columns:
    mn, mx = reservoirs_norm[col].min(), reservoirs_norm[col].max()
    reservoirs_norm[col] = (reservoirs_norm[col] - mn) / (mx - mn)

#plotting all together
plt.figure(figsize=(12, 6))
for col in numerical_columns:
    plt.plot(reservoirs_norm["Date"], reservoirs_norm[col], label=col)
plt.title("Normalized Reservoir Data")
plt.xlabel("Date")
plt.ylabel("Normalized Values")
plt.legend(loc='upper right')
plt.show()
```



```
[12]: #making a prossessed csv file for the data to be used in the streamlit app
reservoirs.to_csv("data/reservoirs_processed.csv", index=False)
```

2 log

I first started with creating a streamlit account and connecting it to my github account. I got some problems beacuse i made 2 seperate repositorys, but i used AI to fix it. From the data set i saw some of the Next_Publication_Date dates where set to 1 january year 1 and i removed these dates. I then did some data cleaning and plotted the numerical as line plot and categorical as box plots. Then i tried to plot all of the numerical variables together using min/max scaling but i encountered some problems during and the plot was wrong. I went to the teaching assisant and got it fixed with his help.

With streamlit I made the file app.py to run streamlit. This will not show on the app but it just runs streamlit. I then made 4 pages for the app: First month which i made graphs to show how the data is disrtubuted for the first month. I mostly looked at what was done in the lecture and used the examples from the live pālayground to make this. The analysis page are the page with plots and graphs with a drop down menu if u want to choose a specific date or variable. I struggled a little bit with this page because i tried to use the example from the playground but it didnt work in live for some reason. I Got the drop down menu but it didnt update in the app so i had do toy a little bit around and after some time i got the final result.

The last 2 pages are just the home page with a welcome text and a page i called test just to try out different things but it is empty now. I probably just gonna use this page to try out things in the future assignments aswell.